

Estrutura de Dados

Percursos e Aplicações

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Percorrer Toda a Árvore

Até aqui, os algoritmos tocavam apenas parte da árvore: a altura mede um caminho, a busca visita só o necessário. Um percurso é diferente — visita cada nó exatamente uma vez, sem exceção. A ordem dessa visita não é um detalhe de implementação: ela determina qual pergunta o percurso responde sobre a estrutura.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Em-Ordem: Esquerda, Raiz, Direita

O percurso em-ordem visita primeiro toda a subárvore esquerda, depois a raiz, e só então toda a subárvore direita — e essa regra se repete recursivamente em cada nível. Como a esquerda sempre guarda chaves menores e a direita chaves maiores ou iguais, essa ordem de visita entrega as chaves já organizadas.



Prof. Everaldo Graciliano Souza Ribeiro

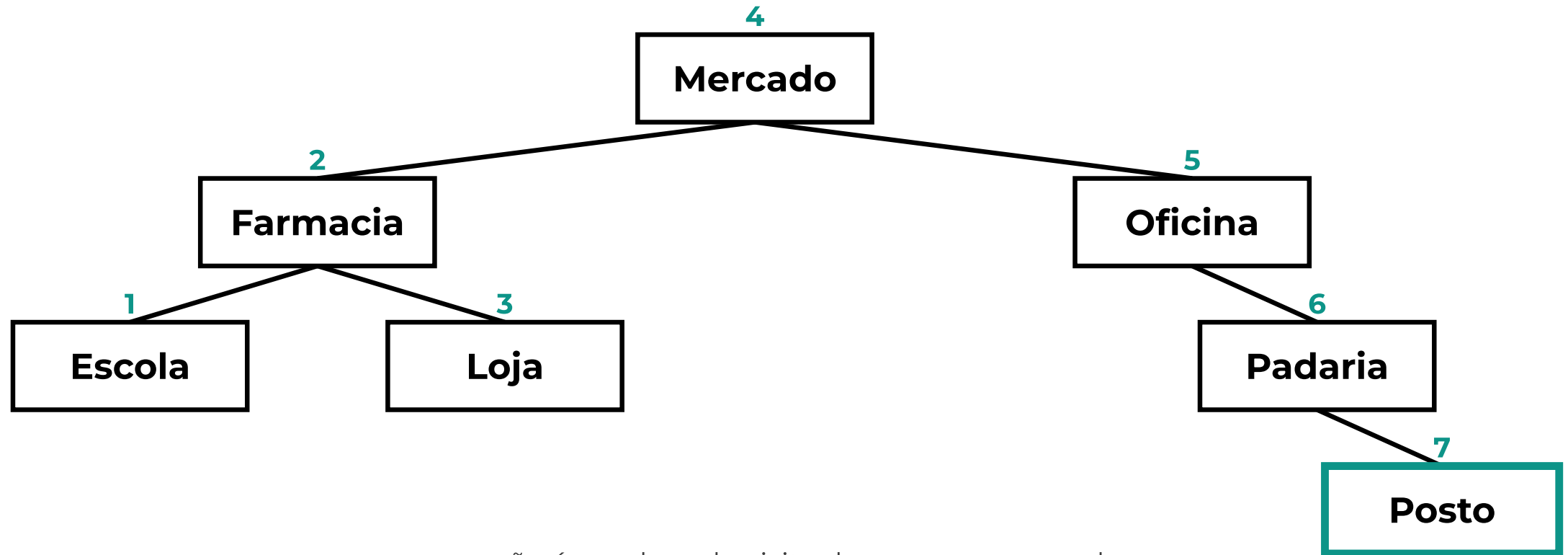
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Em-Ordem: Visita Alfabética



1 a 7 sai **em ordem alfabética exata** — Escola, Farmacia, Loja, Mercado, Oficina, Padaria, Posto.



Em-Ordem em Código

```
def em_ordem(no, saida=None):  
    """Esquerda, raiz, direita."""  
    # Prova, na pratica, que a ABB mantem as chaves ordenadas.  
    if saida is None:  
        saida = []  
    if no is not None:  
        em_ordem(no.esquerda, saida)  
        saida.append(no.valor)  
        em_ordem(no.direita, saida)  
    return saida
```



Em-Ordem Prova a Ordenação da ABB

O índice de pontos da Aula 03 já mantém as chaves ordenadas pela própria regra de inserção. O percurso em-ordem não organiza nada — apenas revela essa ordem já existente, listando os pontos alfabeticamente sem nenhuma etapa extra de ordenação. É a estrutura que garante o resultado, não o algoritmo de percurso.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Pré-Ordem: Raiz, Esquerda, Direita

O percurso pré-ordem visita a raiz antes de qualquer filho, depois percorre a subárvore esquerda inteira e só então a direita. Isso o torna adequado a relatórios que precisam apresentar o todo antes das partes — por exemplo, nomear uma zona de entrega antes de listar os bairros que ela contém.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Pós-Ordem: Esquerda, Direita, Raiz

O percurso pós-ordem inverte a prioridade: visita as duas subárvores inteiras antes de tocar a raiz. É a ordem correta para relatórios de fechamento, em que um nó só pode ser dado como concluído depois que tudo o que está abaixo dele já foi processado — nunca antes, sob pena de inconsistência.



Prof. Everaldo Graciliano Souza Ribeiro

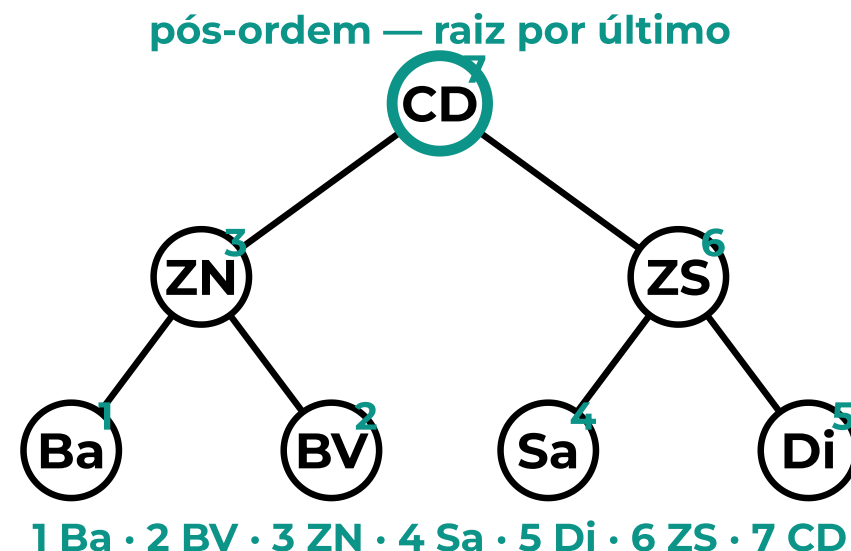
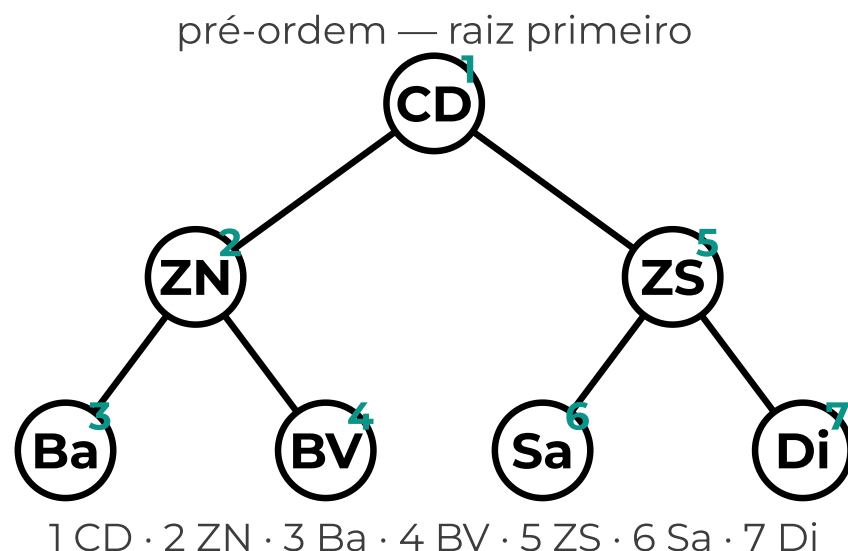
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Pré-Ordem × Pós-Ordem: Opostas



Mesma árvore, duas leituras opostas: **a raiz aparece primeiro ou por último**, nunca no meio.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

everaldoribeiro01@gmail.com

<https://www.linkedin.com/in/everaldoribeiroads>

Pré-Ordem

```
def pre_ordem(no, saida=None):  
    if saida is None:  
        saida = []  
    if no is not None:  
        saida.append(no.valor)           # raiz primeiro  
        pre_ordem(no.esquerda, saida)  
        pre_ordem(no.direita, saida)  
    return saida
```



Pós-Ordem

```
def pos_ordem(no, saida=None):  
    if saida is None:  
        saida = []  
    if no is not None:  
        pos_ordem(no.esquerda, saida)  
        pos_ordem(no.direita, saida)  
        saida.append(no.valor)      # raiz por ultimo  
    return saida
```



Mesma Árvore, Três Leituras

Em-ordem, pré-ordem e pós-ordem percorrem exatamente os mesmos nós, com o mesmo custo proporcional ao total de elementos — o que muda é apenas a ordem em que cada nó é visitado. A estrutura permanece intacta; o que se altera é a pergunta que o relatório precisa responder sobre ela.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Onde os Percursos Entram no LogiRota

No LogiRota, o em-ordem produz a listagem alfabética de pontos exigida por qualquer catálogo de entregas. O pré-ordem e o pós-ordem, aplicados à hierarquia de zonas da Aula 02, geram dois relatórios distintos: um que apresenta cada zona antes de seus bairros, outro que só fecha a zona depois deles.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Três Relatórios, uma Árvore

```
indice = construir_indice(PONTOS)
for ponto in em_ordem(indice):
    print(ponto)                # ordem alfabetica

for zona in pre_ordem(ZONAS):
    print(zona)                 # topo antes dos bairros

for zona in pos_ordem(ZONAS):
    print(zona)                 # bairro fecha antes da zona
```



Atividades

Exercício 1

Qual dos três percursos, aplicado ao índice de pontos da ABB, dispensa qualquer ordenação posterior? Por quê?

Exercício 2

Na hierarquia de zonas, o que a pré-ordem lista primeiro: a zona ou os bairros que ela contém? E a pós-ordem?

Exercício 3

Os três percursos têm o mesmo custo, proporcional ao total de nós. O que exatamente muda entre eles, se o custo é igual?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Escrever `em_ordem_reverso(no)` — direita, raiz, esquerda — e explicar o que essa ordem lista.
2. Escrever `nivel_a_nivel(no)`, que visita a árvore nível por nível em vez de ramo por ramo. Dica: precisa de uma fila, não de recursão.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>